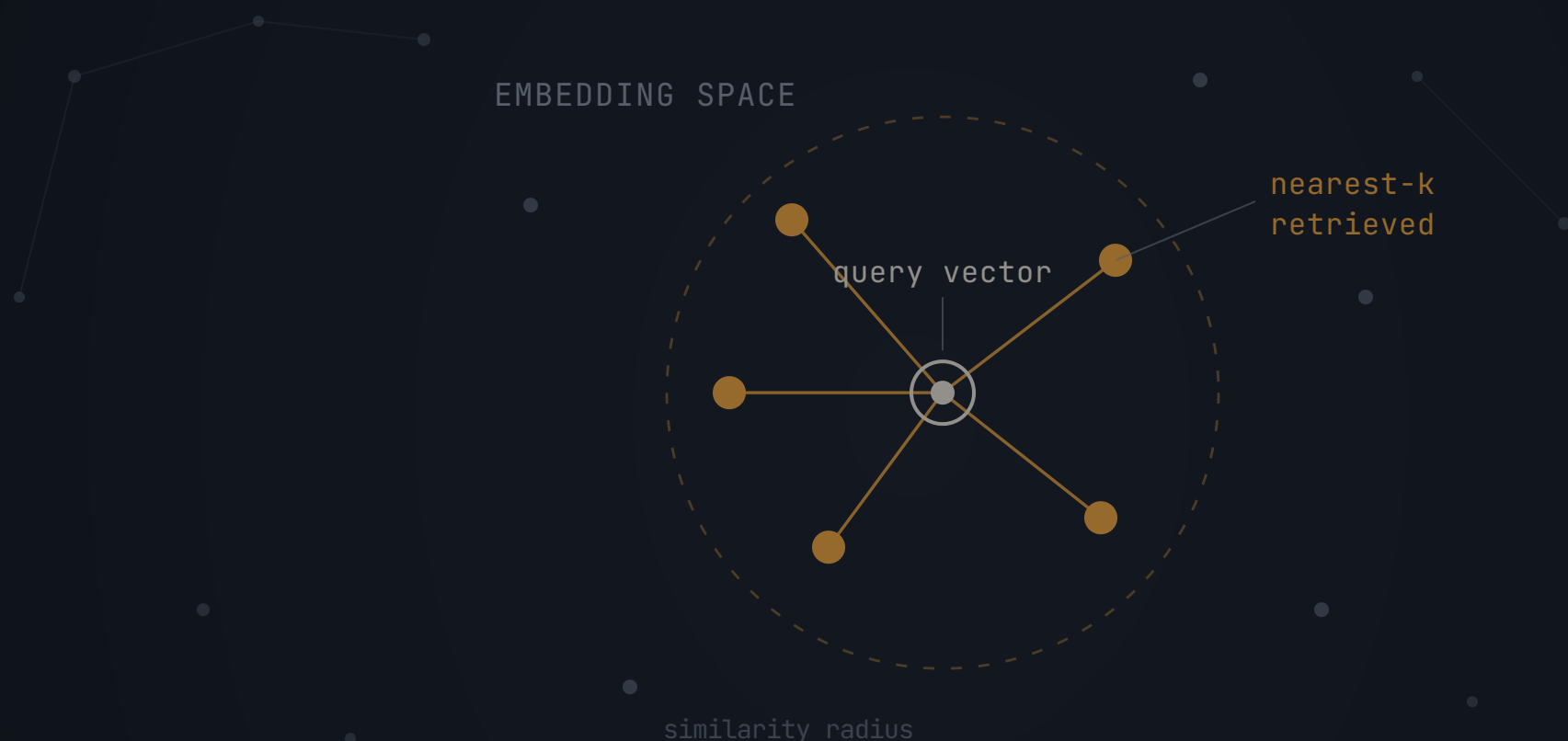


FIG.00 // THE RETRIEVAL EVENT



RAG

RETRIEVAL-AUGMENTED GENERATION

The difference between an LLM that guesses and one that actually knows your data.

R Retrieve **A** Augment **G** Generate



A raw LLM knows the whole internet and nothing about your business.

1/11

It has a training cutoff. It cannot see your documents, your pricing, your policies.

And when it does not know something, it does not stop. It fills the gap with something that sounds right.

For anything customer-facing, a confident wrong answer is worse than no answer.





WHAT RAG ACTUALLY IS

2/11

Give the model the right facts at the moment it answers.

Retrieve the relevant pieces of your data. **Augment** the prompt with them. **Generate** the answer from that.

The model stops answering from memory and starts answering from evidence you handed it.

You are not retraining the model. You are handing it the open book.





Step one: build the index

This happens before anyone asks a question. Build it once, refresh as your data changes.

3/11

SOURCE

Your documents

manuals, policies, tickets, pricing, notes



CHUNK

Split into passages

small enough to be specific, big enough to hold meaning



EMBED

Turn each chunk into a vector

a list of numbers that captures what it means



STORE

Vector database

the searchable index





Step two: answer the question

This runs every time a question comes in.

4/11

QUESTION IN

Embed it the same way



SEARCH

Find the closest chunks

nearest vectors = most relevant passages



AUGMENT

Paste top matches into the prompt



GENERATE

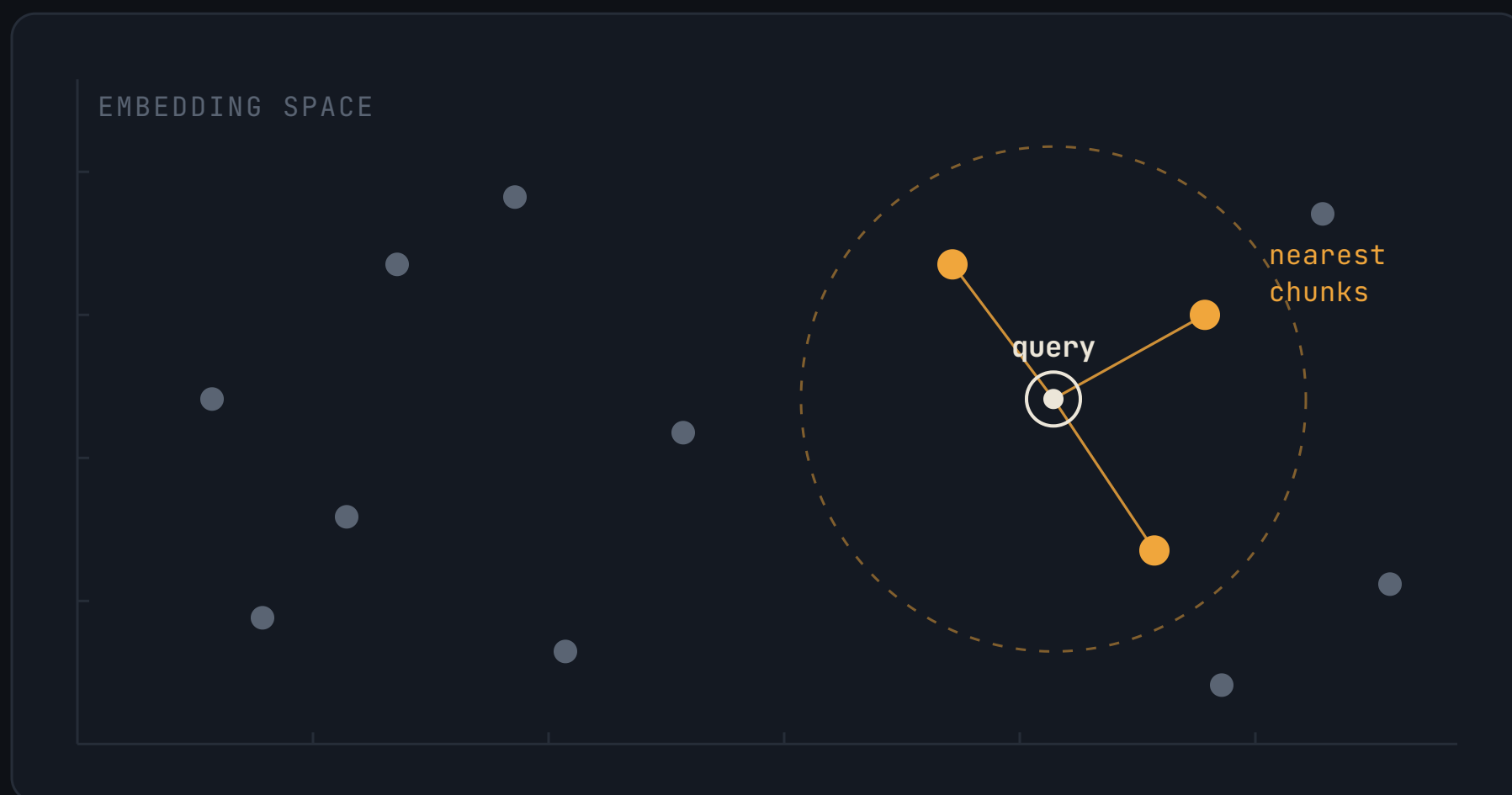
Model answers from that context



WHY VECTORS, NOT KEYWORDS

Keyword search matches words. Vector search matches meaning.

5/11



"after-hours policy" lands next to **"emergency calls outside business hours"** with zero shared words. Closeness in space is closeness in meaning.

NAIVE RAG

The baseline. And a real one.



QUERY - RETRIEVE - ANSWER

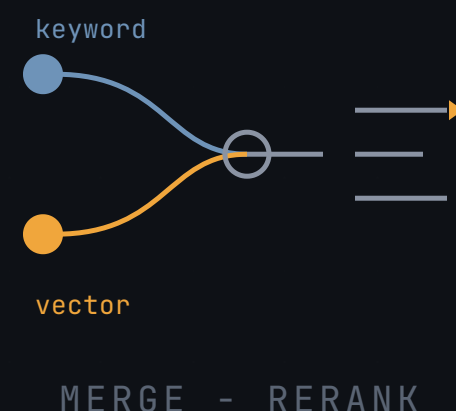
Embed, grab the top matches, stuff them into the prompt. That is it.

Works well when your data is clean and questions are direct.

Breaks when chunks are sized badly, the question is vague, or the answer is spread across several documents.

HYBRID + RERANKING

The best quality-per-effort win.



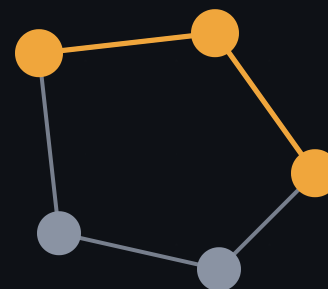
7/11

- + **Hybrid:** keyword and vector search together. Catches exact terms (part numbers, names) pure vectors miss.
- + **Rerank:** pull more candidates than you need, then reorder by true relevance before the prompt.

Most "our RAG is bad" problems are actually retrieval problems. This is where you fix them.

GRAPHRAG

When the answer lives in the connections.



ENTITIES - RELATIONSHIPS

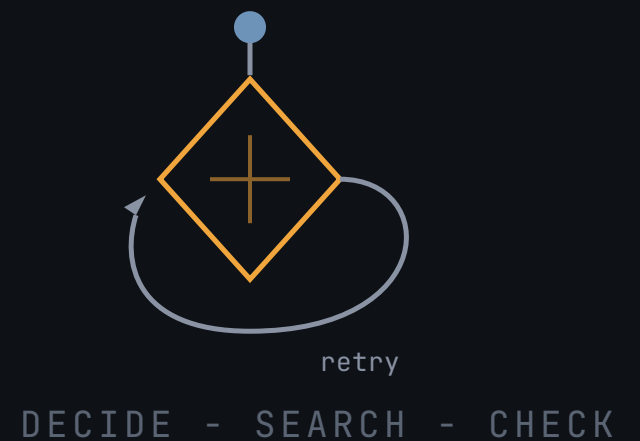
Build a graph of entities and how they relate.
Retrieval walks relationships, not just similarity.

Strong for: "which customers were hit by the outage on the node that Vendor X supplies?"

Heavier to build. Reach for it when the relationships **are** the point.

AGENTIC RAG

The model runs retrieval itself.



9/11

- ? Do I even need to look this up?
- ? What should I actually search for?
- ? Is this enough, or do I search again?

Multi-step. It can hit several sources, check its own answer, and retry.

More capable, more latency, more cost. Match it to the problem, do not default to it.

AN AI RECEPTIONIST THAT ANSWERS CALLS

caller: "do you service heat pumps in my area,
and what's the trip fee?"

The model has no idea. That business's hours, pricing, service area and policies live in **their** knowledge base, not in the model's weights.

RAG pulls that specific business's facts at call time and grounds the answer in them.

Same model, different retrieved context per business. That is how one system serves hundreds of companies without retraining anything.



RAG is only as good as what you retrieve.

11/11

Garbage chunks in, confident garbage out.

The hard work was never the model. It is chunking, retrieval quality, and keeping the index fresh.

Retrieval is the product. The model is just the last step.

What is your data actually shaped like? **That decides which RAG you need.**

